

Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 6 : sous-catégories

1. Ajouter les sous-catégories de produits.

Créez l'entité « SubCategory » : `symfony console make:entity subCategory`

```
PS C:\Symfony\e-commerce> symfony console make:entity subCategory
created: src/Entity/SubCategory.php
created: src/Repository/SubCategoryRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.
```

On ajoute ensuite le champ « name » (string, 255 caractères, Non Null) :

```
New property name (press <return> to stop adding fields):
> name

Field type (enter ? to see all types) [string]:
>

Field length [255]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/SubCategory.php
```

Puis une nouvelle propriété pour faire lien entre la table « Category » (déjà existante) et la table « SubCategory ». On choisit le type « ManyToOne » (plusieurs Sous-catégories appartiennent à une et une seule catégorie) :

```
Add another property? Enter the property name (or press <return> to stop adding fields):
> category

Field type (enter ? to see all types) [string]:
> ManyToOne

What class should this entity be related to?:
> Category

Is the SubCategory.category property allowed to be null (nullable)? (yes/no) [yes]:
> no

Do you want to add a new property to Category so that you can access/update SubCategory objects from it - e.g. $category->getSubCategories()? (yes/no) [yes]:
>

A new property will also be added to the Category class so that you can access the related SubCategory objects from it.

New field name inside Category [subCategories]:
>
```

Attention, les sous-catégories doivent être supprimées si la catégorie n'existe plus. Répondez « yes » à la question « Do you want to automatically delete orphaned App\Entity\SubCategory objects (orphanRemoval) ? » :

```

Do you want to activate orphanRemoval on your relationship?
A SubCategory is "orphaned" when it is removed from its related Category.
e.g. $category->removeSubCategory($subCategory)

NOTE: If a SubCategory may *change* from one Category to another, answer "no".

Do you want to automatically delete orphaned App\Entity\SubCategory objects (orphanRemoval)? (yes/no) [no]:
> yes

updated: src/Entity/SubCategory.php
updated: src/Entity/Category.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

Success!

Next: When you're ready, create a migration with symfony.exe console make:migration

```

2. Préparer la migration pour créer la table « SubCategory » dans la BDD

`symfony console make:migration`

Exécuter ensuite la migration avec doctrine :

`symfony console doctrine:migrations:migrate`

Vous pouvez voir le résultat dans phpMyAdmin avec la clé étrangère « category_id » qui a bien été créée.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
☐ 1	id	int(11)			Non	Aucun(e)		AUTO_INCREMENT	Modifier Supprimer Plus
☐ 2	category_id	int(11)			Non	Aucun(e)			Modifier Supprimer Plus
☐ 3	name	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)			Modifier Supprimer Plus

3. Créez le CRUD (Create, Read, Update, Delete) pour l'entité « SubCategory » :

`symfony console make:crud`

```
PS C:\Symfony\e-commerce> symfony console make:crud

The class name of the entity to create CRUD (e.g. GrumpyGnome):
> SubCategory

Choose a name for your controller class (e.g. SubCategoryController) [SubCategoryController]:
>

Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
>

created: src/Controller/SubCategoryController.php
created: src/Form/SubCategoryForm.php
created: templates/sub_category/_delete_form.html.twig
created: templates/sub_category/_form.html.twig
created: templates/sub_category/edit.html.twig
created: templates/sub_category/index.html.twig
created: templates/sub_category/new.html.twig
created: templates/sub_category/show.html.twig

Success!

Next: Check your new CRUD by going to /sub/category/
```

4. Modifier la vue « templates\sub_category\index.html.twig » pour mettre en forme avec Bootstrap :

```
{% extends 'base.html.twig' %}

{% block title %}Liste des sous-catégories{% endblock %}

{% block body %}
<div class = "container">
    <br>
    <h1>Sous-catégories</h1>
    <br>
    {% include 'layouts/_flash_message.html.twig' %}
    <table class="table">
        <thead>
            <tr>
                <th>Id</th>
                <th>Sous-catégorie</th>
                <th>Catégorie</th>
                <th>actions</th>
            </tr>
        </thead>
        <tbody>
            {% for sub_category in sub_categories %}
                <tr>
                    <td>{{ sub_category.id }}</td>
                    <td>{{ sub_category.name }}</td>
                    <td>{{ sub_category.category.name }}</td>
```

```

        <td>
            <a class="btn btn-outline-primary" href="{ { path
('app_sub_category_show', { 'id': sub_category.id }) } }">Afficher</a>
            <a class="btn btn-outline-secondary" href="{ { path
('app_sub_category_edit', { 'id': sub_category.id }) } }">Modifier</a>
        </td>
    </tr>
{% else %}
    <tr>
        <td colspan="3">Aucune sous-catégorie à afficher</td>
    </tr>
{% endfor %}
</tbody>
</table>

    <a class="btn btn-outline-success" href="{ { path('app_sub_category_new') } }">
    Nouvelle sous-catégorie</a>
</div>
{% endblock %}

```

5. Mettez en forme le formulaire dans le fichier « templates\sub_category_form.html.twig » :

```

{{ form_start(form) }}
    <label for="">Nom de la sous-catégorie : </label>
    {{ form_widget(form.name, { 'attr': { 'class': 'form form-control', 'placeholder': "nom" } }) }}
    <br>
    <label for="">Catégorie : </label>
    {{ form_widget(form.category, { 'attr': { 'class': 'form form-control' } }) }}
    <br>
    <button class="btn btn-primary">{{ button_label|default('Enregistrer') }}</button>
{{ form_end(form) }}

```

6. Modifiez la fonction « buildForm » dans le fichier « src\Form\SubCategoryForm.php » pour afficher le label « name » quand on remplira la liste déroulante des catégories :

```

public function buildForm(FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('name')
        ->add('category', EntityType::class, [
            'class' => Category::class,
            'choice_label' => 'name',
        ])
    ;
}

```

7. Modifiez « templates\sub_category\delete_form.html.twig » :

```
<form method="post" action="{% path('app_sub_category_delete', {'id': sub_category.id}) %}"
onsubmit="return confirm('Voulez-vous vraiment supprimer cette sous-catégorie ?');">
<input type="hidden" name="_token" value="{% csrf_token('delete' ~ sub_category.id) %}">
    <button class="btn btn-danger">Supprimer</button>
</form>
```

8. Modifiez « templates\sub_category\edit.html.twig » :

```
{% extends 'base.html.twig' %}

{% block title %}Modification sous-catégorie{% endblock %}

{% block body %}
<div class = "container">
    <h1>Modifier une sous-catégorie</h1>

    {{ include('sub_category/_form.html.twig', {'button_label': 'Modifier'}) }}
    <br>
    {{ include('sub_category/_delete_form.html.twig') }}
    <br>
    <a class="btn btn-dark" href="{% path('app_sub_category_index') %}">Retour</a>
</div>
{% endblock %}
```

9. Modifiez « templates\sub_category\new.html.twig » :

```
{% extends 'base.html.twig' %}

{% block title %}Ajout sous-catégorie{% endblock %}

{% block body %}
<div class = "container">
    <br>
    <h1>Ajouter une sous-catégorie</h1>
    <br>
    {{ include('sub_category/_form.html.twig') }}
    <br>
    <a class="btn btn-dark" href="{% path('app_sub_category_index') %}">Retour</a>
</div>
{% endblock %}
```

10. Modifiez « templates\sub_category\show.html.twig » :

```
{% extends 'base.html.twig' %}

{% block title %}SubCategory{% endblock %}
```

```
{% block body %}
    <div class = "container">
        <h1>Sous-catégorie</h1>

        <table class="table">
            <tbody>
                <tr>
                    <th>Id</th>
                    <td>{{ sub_category.id }}</td>
                </tr>
                <tr>
                    <th>Sous-catégorie</th>
                    <td>{{ sub_category.name }}</td>
                </tr>
                <tr>
                    <th>Catégorie</th>
                    <td>{{ sub_category.category.name }}</td>
                </tr>
            </tbody>
        </table>

        <a class="btn btn-outline-primary" href="{{ path('app_sub_category_edit', {'id':
sub_category.id}) }}">Modifier</a>
        <br><br>
        {{ include('sub_category/_delete_form.html.twig') }}
        <br>
        <a class="btn btn-dark" href="{{ path('app_sub_category_index') }}">Retour</a>
    </div>
{% endblock %}
```

TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Réserver la gestion des sous-catégories aux utilisateurs ayant le rôle « ROLE_ADMIN ».
- Ajouter la gestion des sous-catégories dans la navbar (uniquement pour les administrateurs).
- Ajouter des messages flash lors de l'ajout, la modification et la suppression d'une sous-catégorie.